Algorithmique & Programmation

Corwin FEVRE corwin.fevre@eigsi.fr

FISA 3 — EIGSI



Inspiré du cours de M. Thomas RAIMBAULT

Partie III

Récursivité et Complexité

1. La récursivité
2. Complexité

1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

Préambule

Un des intérêts à utiliser une machine, c'est d'effectuer des **tâches répétitives** !

- pour cela les **instructions itératives** sont adaptées
  $\Leftrightarrow$ les boucles
  - La boucle `for` est adaptée lorsque le nombre d'itérations est connu à l'avance.
  - La boucle `while` de façon générale.

✎ Dans tous les cas, une condition d'arrêt est nécessaire pour sortir de la boucle.

Code (exemple 1)

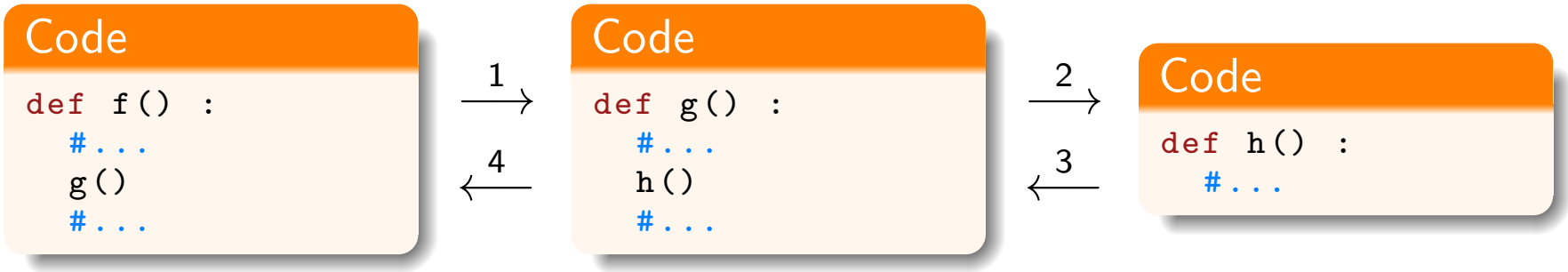
```
1 n = input()
2 for i in range(n) :
3     # ici ça tourne...
4     # n fois au plus
```

Code (exemple 2)

```
1 n = input()
2 while n > 0 :
3   #ici ça tourne...
4   n = input()
```

Pour le moment nous avons vu l'approche itérative (avec des boucles) mais selon la nature du problème, il est possible d'effectuer des tâches répétitives à l'aide de **fonctions récursives**.

Remarque préalable



Dans le code ci-dessus :

1. L'exécution de la fonction `f` a besoin de la fonction g, on dit que « `f` fait appel à g ».
2. Cette fonction g fait elle-même appel à la fonction `h`.
3. Lorsque l'exécution de `h` est terminée, celle de g peut se poursuivre,
4. et lorsque l'exécution de g est terminée, celle de `h` peut se poursuivre et finir.

1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

Exemple introductif

Calculons la factorielle d'un nombre n.

- $n! = n \times n-1 \times n-2 \times \ldots \times 3 \times 2 \times 1$
- $n! = n \times (n-1)!$
- $1! = 1$
- $0! = 1$ par convention

Exemple introductif

Écrivons cette fonction pas à pas :

D'abord, on initialise un résultat et son retour.

Code

```
1 # pré-requis : n >= 0
2 def factorielle(n) :
3   resultat = 0
4
5   #...
6
7   return resultat
```

Exemple introductif

Ensuite, on y ajoute les cas spécifiques, qui serviront de conditions d'arrêt de la récursivité.

Code

```
1  # pré-requis : n >= 0
2  def factorielle(n):
3    resultat = 0
4
5    if( n == 0 ):
6      resultat = 1
7
8    else:
9      if( n == 1 ):
10       resultat = 1
11
12     else:
13       # cas général
14
15
16   return resultat
```

Exemple introductif

On continue avec le cas général :

Code

```
1  # pré-requis : n >= 0
2  def factorielle(n):
3    resultat = 0
4
5    if( n == 0 ):
6      resultat = 1
7
8    else:
9      if( n == 1 ):
10       resultat = 1
11
12     else: # cas général
13       resultat = n * ...
14
15
16   return resultat
```

Exemple introductif

On appelle une autre fonction pour calculer le factorielle de (n-1)

Code

```
1  # pré-requis : n >= 0
2  def factorielle(n):
3    resultat = 0
4
5    if( n == 0 ):
6      resultat = 1
7
8    else:
9      if( n == 1 ):
10       resultat = 1
11
12     else:
13       resultat = n * autreFonctionFactorielle(n-1)
14     #               besoin d'une autre fonction pour (n-1)!
15
16
17   return resultat
```

Exemple introductif

L'autre fonction, qui ressemble étrangement à la première... appelle encore une autre fonction, toujours aussi semblable...

Code

```
1  # pour m >= 0
2  def autreFonctionFactorielle(m):
3    resultat = 0
4
5    if( m == 0 ):
6      resultat = 1
7
8    else:
9      if( m == 1 ):
10       resultat = 1
11
12     else:
13       resultat = m * encoreAutreFonctionFactorielle(m-1)
14     #              on délègue pour calculer la suite...
15
16
17   return resultat
```

Exemple introductif

Finalement, le code des fonctions :

- `factorielle()`
- `autreFonctionFactorielle()`
- `encoreAutreFonctionFactorielle()`
- ...

est le même ! ! !

On a donc pas à redéfinir une nouvelle fonction à chaque fois, on peut simplement appeler la même fonction à chaque fois !

On est dans le cas d'une fonction récursive.

1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

Définition (Fonction récursive)

Une fonction est dite récursive lorsqu'elle s'appelle elle-même.

✎ à noter qu'il s'agit à chaque fois d'une exécution bien distincte de chaque fonction appelée, c'est ce l'on appelle **une instance de fonction**.

Intérêt de la récursivité

- Facilité de mise en œuvre lorsque la nature du sous-problème est de même nature que le problème principal.
- Exemples
  - suite de fibonacci
  - recherche par dichotomie
  - tri rapide (*Quick sort*)
  - parcours d'un graphe
  - *etc.*

Exemple – La factorielle <1/3>

$4! = 4 \times 3 \times 2 \times 1$

Ce qui peut se réécrire : $4! = 4 \times 3!$

- avec $3! = 3 \times 2!$
  - avec $2! = 2 \times 1!$
    - avec $1! = 1$ $\leftarrow$ fin du calcul

C'est à dire : $4 \times (3 \times (2 \times (1)))$

Donc finalement,

- résoudre le problème de 4! c'est être capable de résoudre le problème de 3!,
- qui lui-même est le problème de résoudre 2!,
- qui ...,
- pour finir avec $1! = 1$ par définition.

Exemple – La factorielle <2/3>

Code

```
1  # pré-requis : n >= 0
2  def factorielle(n):
3    resultat = 0
4
5    if( n == 0 ):
6      resultat = 1 # condition d'arrêt 0! = 1
7
8    else:
9      if( n == 1 ):
10       resultat = 1 # condition d'arrêt 1! = 1
11
12     else:
13       resultat = n * factorielle(n - 1) # cas général
14
15
16
17   return resultat
```

Exemple – La factorielle <3/3>

Code

```
1  # pré-requis : n >= 0
2  def factorielle(n):
3    if( n == 0 ):
4      return 1 # condition d'arrêt 0! = 1
5
6    if( n == 1 ):
7      return 1 # condition d'arrêt 1! = 1
8
9    return n * factorielle(n - 1) # cas général
```

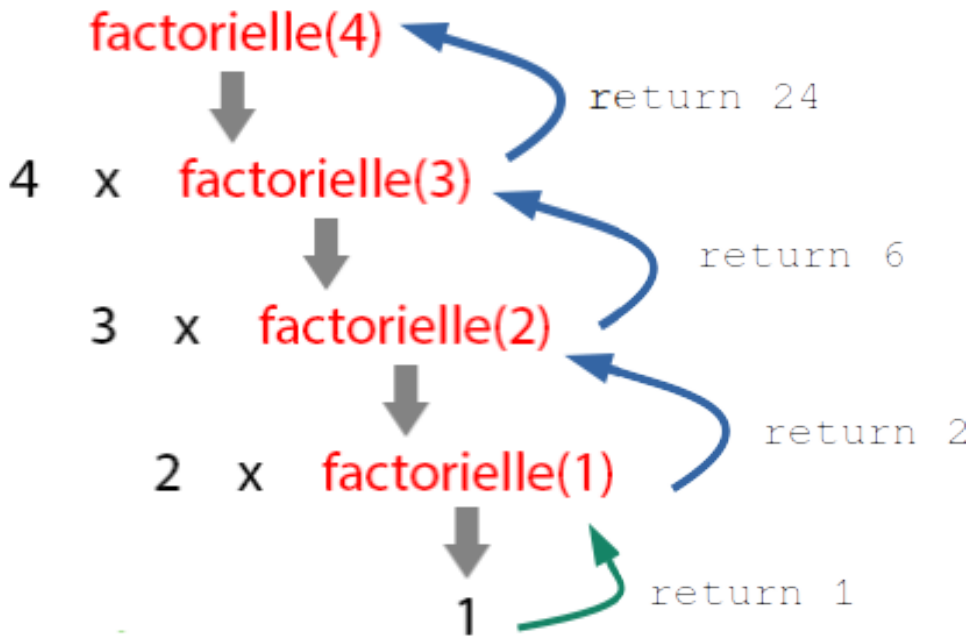
Conseils pour coder une fonction récursive :

- Traiter les conditions d'arrêt en premier,
  - → cela permet de ne pas les oublier !
  - → avec le mot-clé `return` cela simplifie le code (pas besoin de `else`) et le rend plus lisible (profondeur d'indentation limitée).

✎ Donc plusieurs mot-clés `return` préconisés avec une fonction récursive, un par cas (général ou d'arrêt).

Exemple – La factorielle <3/3>

Illustration à l'exécution :



1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

Itératif *versus* Récursif

Ci-dessous une approche récursive et une approche itérative pour le calcul de la factorielle.

Code (version récursive)

```
1 # pour n >= 0
2 def factRec(n):
3
4   if(n <= 1):
5     return 1 #0! = 1! =1
6
7   return n * factRec(n-1)
```

Code (version itérative)

```
1 # pour n >= 0
2 def factIter(n):
3
4   res = 1 #0! = 1! =1
5   while(n > 1):
6     res *= n
7     n = n-1
8
9   return res
```

Itératif *versus* Récursif

Ci-dessous une approche récursive et une approche itérative pour le calcul de la factorielle.

Code (version récursive)

```
1 # pour n >= 0
2 def factRec(n):
3
4   if(n <= 1):
5     return 1 #0! = 1! =1
6
7   return n * factRec(n-1)
```

Code (version itérative)

```
1 # pour n >= 0
2 def factIter(n):
3
4   res = 1 #0! = 1! =1
5   while(n > 1):
6     res *= n
7     n = n-1
8
9   return res
```

D'un point de vue à l'exécution :

Itératif *versus* Récursif

Ci-dessous une approche récursive et une approche itérative pour le calcul de la factorielle.

Code (version récursive)

```
1 # pour n >= 0
2 def factRec(n):
3
4   if(n <= 1):
5     return 1 #0! = 1! =1
6
7   return n * factRec(n-1)
```

Code (version itérative)

```
1 # pour n >= 0
2 def factIter(n):
3
4   res = 1 #0! = 1! =1
5   while(n > 1):
6     res *= n
7     n = n-1
8
9   return res
```

D'un point de vue à l'exécution :

`factRec(4) = 4 * (3 * (2 * (1)))`

`factIter(4) = 1 * 4 * 3 * 2`

Bilan – Itératif *versus* Récursif

Équivalence du pouvoir expressif entre les deux approches récursive ou itérative

- Donc, tout algorithme itératif peut s'écrire en récursif et vice-versa.
- Selon le problème, une approche est plus simple à concevoir et coder qu'une autre :)

Cela dit, la façon même de concevoir et de mettre en œuvre est assez différente entre les deux approches !

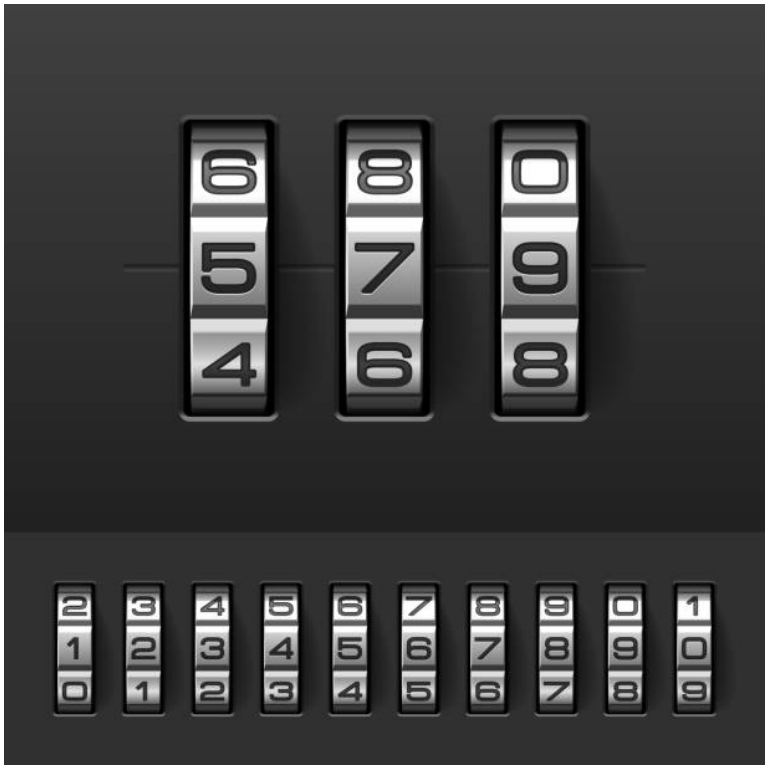
1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

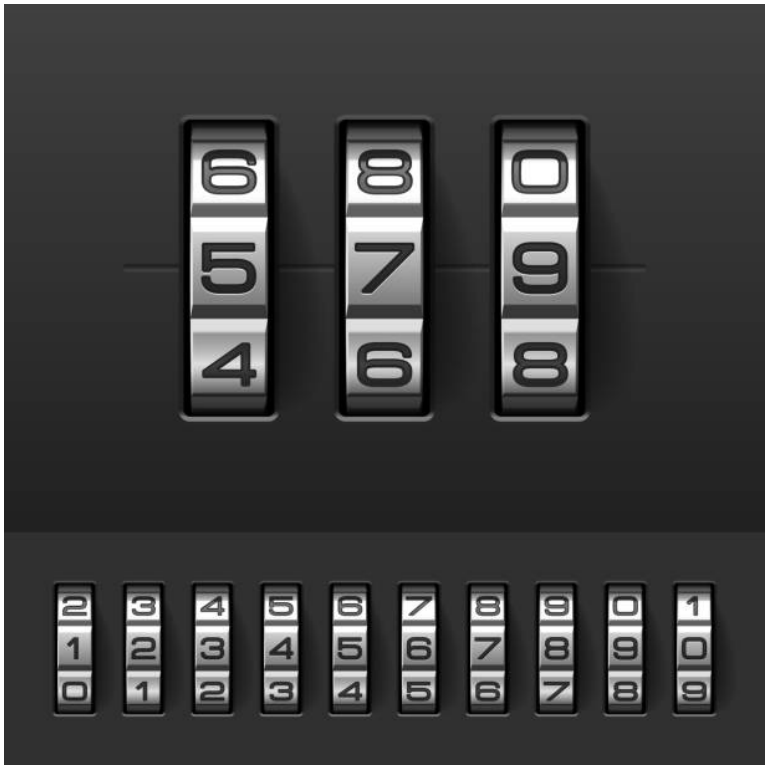
2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

Exemple préliminaire - Une histoire de combinaison



J'ai oublié mon code, comment faire pour le retrouver ?

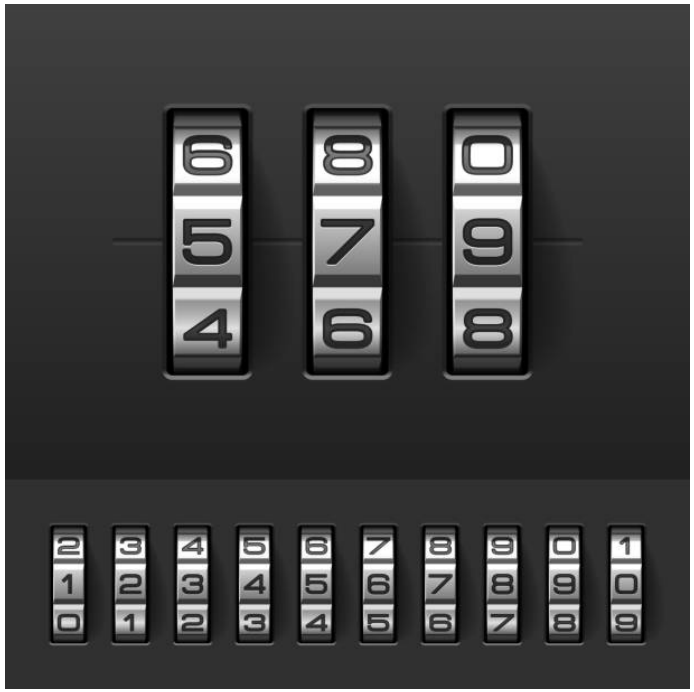
Exemple préliminaire - Une histoire de combinaison



J'ai oublié mon code, comment faire pour le retrouver ?

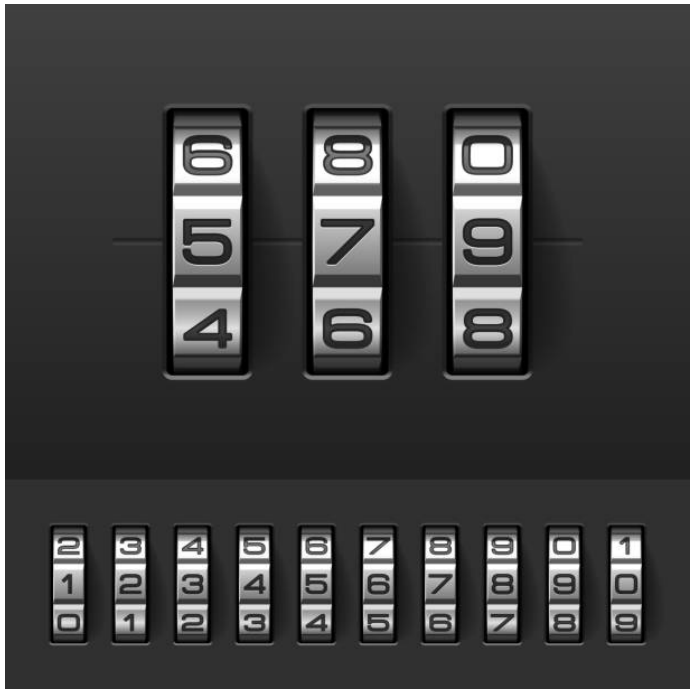
En testant toutes les combinaisons, c'est à dire en faisant une **recherche exhaustive** (ou brute force).

Cadenas à trois chiffres - à la main



Combien y à t'il de combinaisons ?

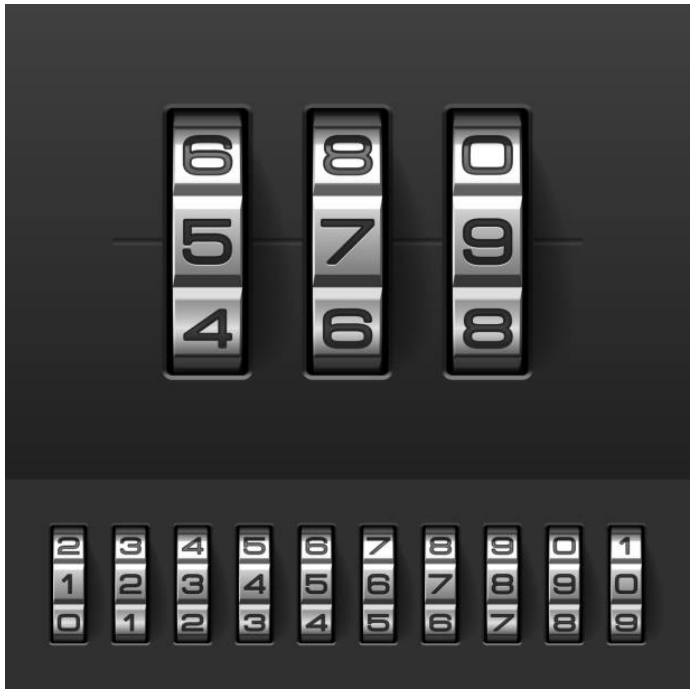
Cadenas à trois chiffres - à la main



Combien y à t'il de combinaisons ?

Il y a **10** chiffres possibles et **3** chiffres à positionner.

Cadenas à trois chiffres - à la main

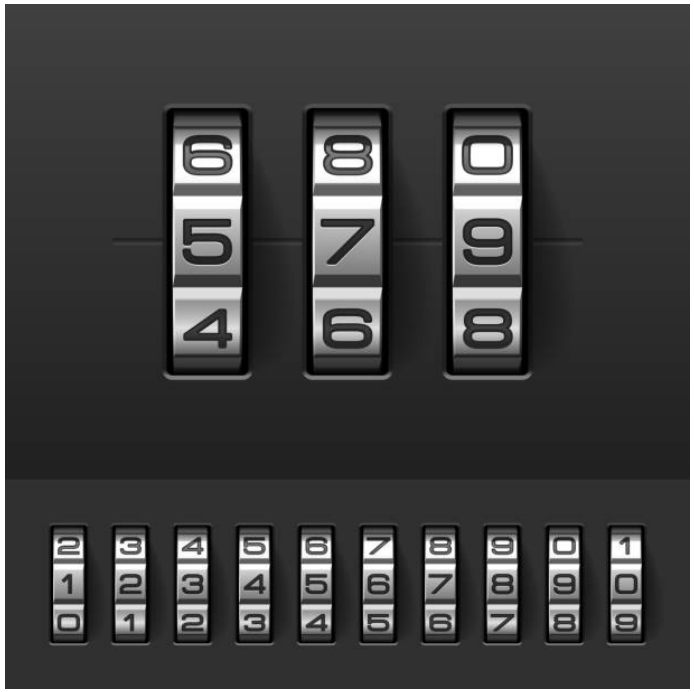


Combien y à t'il de combinaisons ?

Il y a **10** chiffres possibles et **3** chiffres à positionner.

$\Rightarrow 10 * 10 * 10 = 10^3$

Cadenas à trois chiffres - à la main



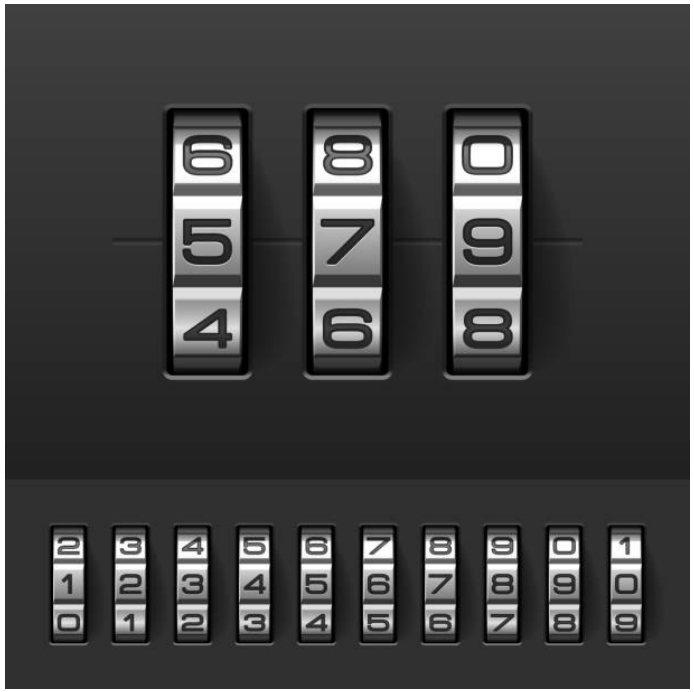
Combien y à t'il de combinaisons ?

Il y a **10** chiffres possibles et **3** chiffres à positionner.

$\Rightarrow 10 * 10 * 10 = 10^3$

Combien de temps pour trouver le code à la main ?

Cadenas à trois chiffres - à la main



Combien y à t'il de combinaisons ?

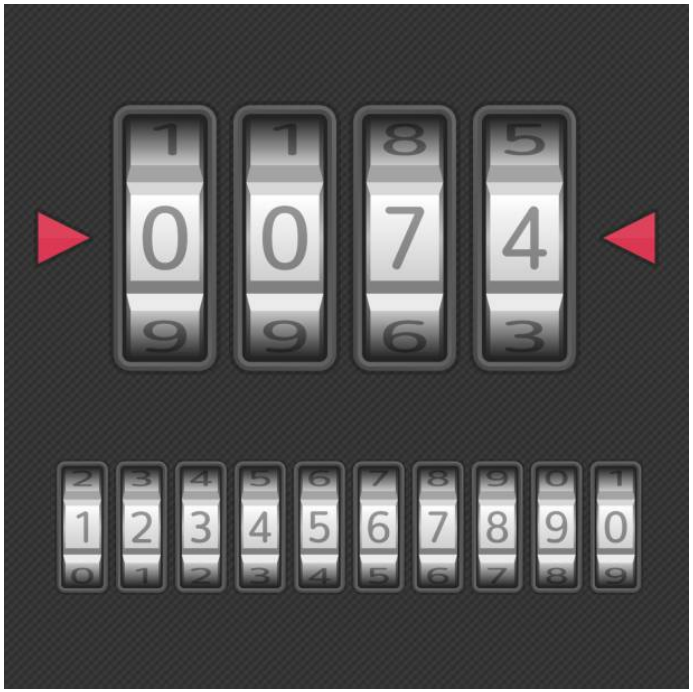
Il y a **10** chiffres possibles et **3** chiffres à positionner.

$\Rightarrow 10 * 10 * 10 = 10^3$

Combien de temps pour trouver le code à la main ?

$\approx$ 2 sec par combinaison
$2 * 1000/60 = 33.33333$
$\approx 30 minutes$

Cadenas à quatre chiffres

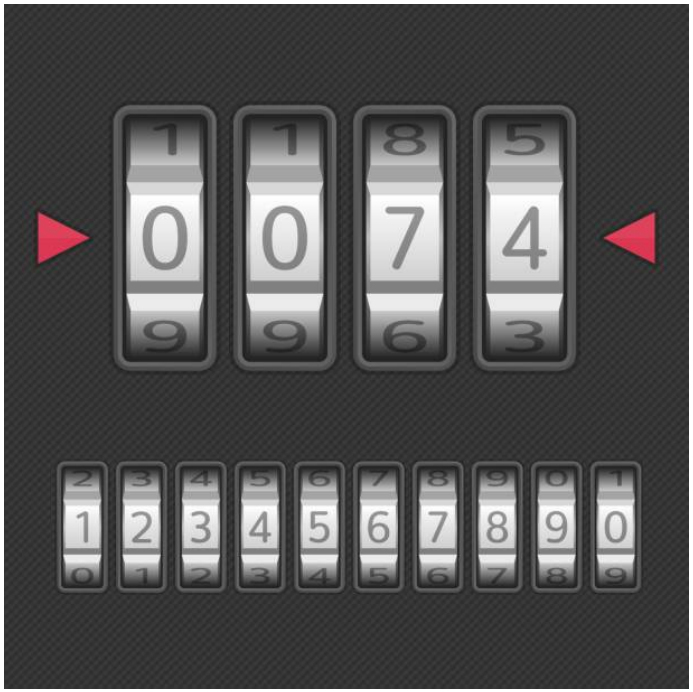


Combien y à t'il de combinaisons ?

Il y a **10** chiffres possibles et **4** chiffres à positionner.

$\Rightarrow 10 * 10 * 10 * 10 = 10^4$

Cadenas à quatre chiffres



Combien y à t'il de combinaisons ?

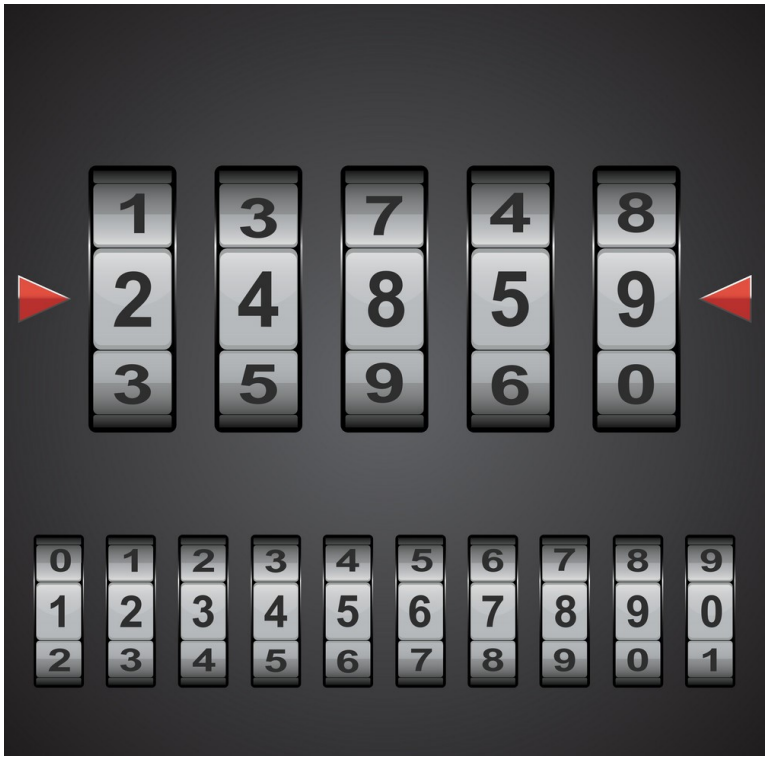
Il y a **10** chiffres possibles et **4** chiffres à positionner.

$\Rightarrow 10 * 10 * 10 * 10 = 10^4$

Combien de temps pour trouver le code à la main ?

$\approx$ 2 sec par combinaison
$2 * 10000/60/60 = 5.55555$
$\approx 5 - 6 heures$

Cadenas à cinq chiffres

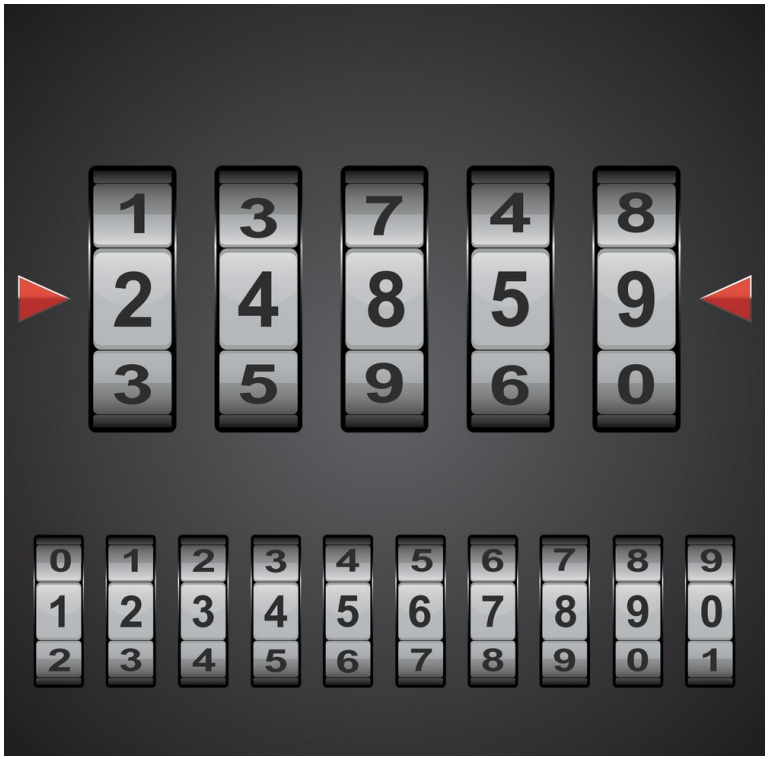


Combien y à t'il de combinaisons ?

Il y a **10** chiffres possibles et **5** chiffres à positionner.

$\Rightarrow 10 * 10 * 10 * 10 * 10 = 10^5$

Cadenas à cinq chiffres



Combien y à t'il de combinaisons ?

Il y a **10** chiffres possibles et **5** chiffres à positionner.

$\Rightarrow 10 * 10 * 10 * 10 * 10 = 10^5$

Combien de temps pour trouver le code à la main ?

$\approx$ 2 sec par combinaison
$2 * 100000/60/60 = 55.555$
$\approx 55 heures, 2 - 3 jours$

Et par un ordinateur ?

Un ordinateur prend 0.001 µs par opération simple (changer le code du cadenas).

Complexité	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
$n = 10$	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	4 s
$n = 30$	< 1 s	< 1 s	< 1 s	< 1 s	18 min	10^{25} ans
$n = 50$	< 1 s	< 1 s	< 1 s	< 1 s	36 ans	∞
$n = 100$	< 1 s	< 1 s	< 1 s	1s	10^{17} ans	∞
$n = 1000$	< 1 s	< 1 s	1s	18 min	∞	∞
$n = 10000$	< 1 s	< 1 s	2 min	12 jours	∞	∞
$n = 100000$	< 1 s	2 s	3 heures	32 ans	∞	∞
$n = 1000000$	1s	20s	12 jours	31,710 ans	∞	∞

Où se positionne t-on avec notre problème du cadenas (10^n) ?

Et par un ordinateur ?

Complexité	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
$n = 10$	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	4 s
$n = 30$	< 1 s	< 1 s	< 1 s	< 1 s	18 min	10^{25} ans
$n = 50$	< 1 s	< 1 s	< 1 s	< 1 s	36 ans	∞
$n = 100$	< 1 s	< 1 s	< 1 s	1s	10^{17} ans	∞
$n = 1000$	< 1 s	< 1 s	1s	18 min	∞	∞
$n = 10000$	< 1 s	< 1 s	2 min	12 jours	∞	∞
$n = 100000$	< 1 s	2 s	3 heures	32 ans	∞	∞
$n = 1000000$	1s	20s	12 jours	31,710 ans	∞	∞

En complexité exponentielle !

Information

La complexité en temps d'un algorithme, c'est finalement le nombre d'opérations nécessaires pour résoudre un problème.

1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

Définition

Définition

Le calcul de la complexité d'un algorithme permet de mesurer sa performance de résolution d'un problème. Elle mesure le nombre d'opérations fondamentales à effectuer.

Opération fondamentale : assignation, comparaison, opération arithmétique...

Définition

Définition

Le calcul de la complexité d'un algorithme permet de mesurer sa performance de résolution d'un problème. Elle mesure le nombre d'opérations fondamentales à effectuer.

Opération fondamentale : assignation, comparaison, opération arithmétique...

On ne mesure pas le temps ou l'espace exact nécessaire pour l'exécution d'un programme :

- il a été exécuté sur une machine en particulier
- tout le monde n'a pas la même machine ou le même matériel

Temps et espace

Il existe deux types de complexité :

- **complexité spatiale :** permet de quantifier l'utilisation de la mémoire
- **complexité temporelle :** permet de quantifier la vitesse d'exécution

Tous les algorithmes ne se valent pas !

Deux critères pour les évaluer :

- Temps de calcul : lent *vs* rapide
- Espace (mémoire) utilisé : peu *vs* beaucoup

Complexité algorithmique

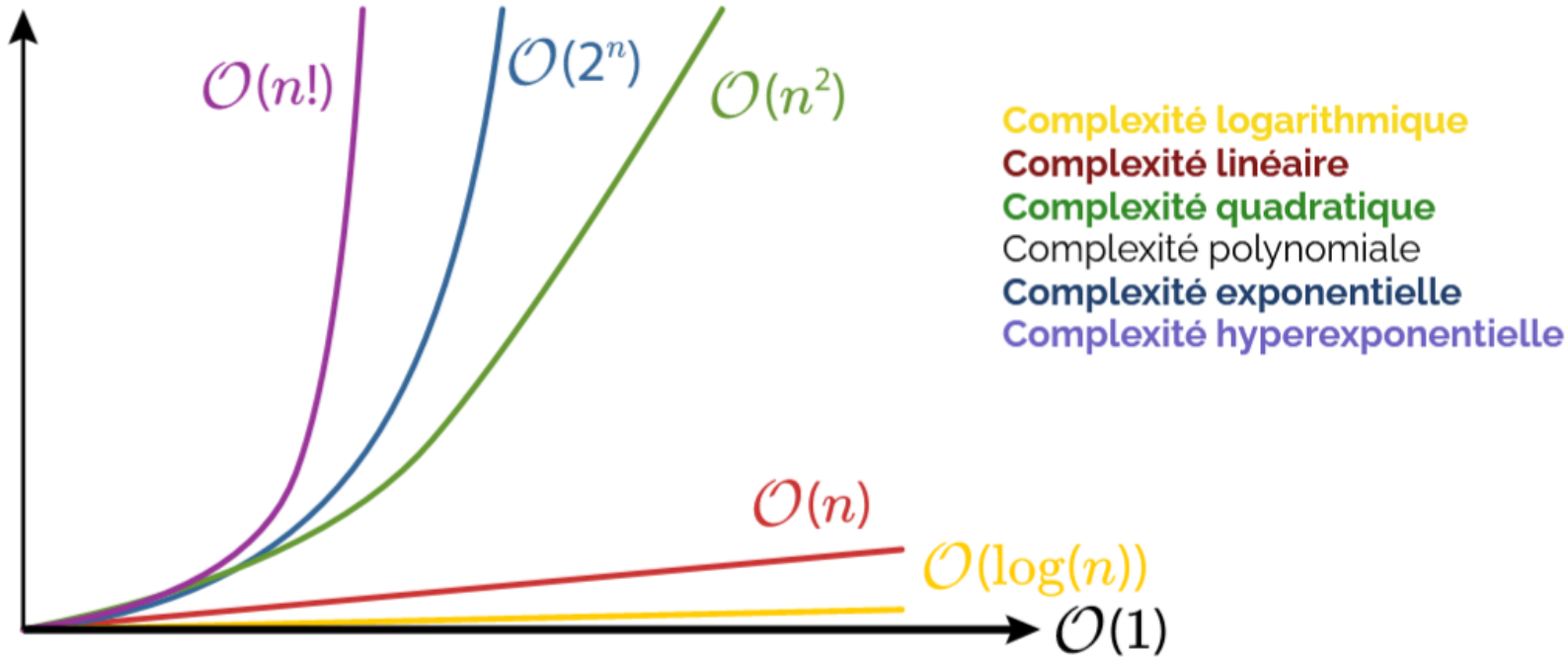
Exemple d'algorithmes et de leurs complexités :

Famille d'algorithmes	Notation	Exemples
Algorithmes **constants**	$\Theta(1)$	Echange deux valeurs
Algorithmes **logarithmiques**	$\Theta(\log n)$	Recherche binaire (dichotomique)
Algorithmes **linéaires**	$\Theta(n)$	Recherche séquentielle
Algorithmes **quasi-linéaires**	$\Theta(n \log n)$	Tri par fusion
Algorithmes **quadratiques**	$\Theta(n^2)$	Tri par sélection
Algorithmes **cubiques**	$\Theta(n^3)$	Produit de deux matrices
Algorithmes **polynomiaux**	$\Theta(n^p),\ p \geq 1$	
Algorithmes **exponentiels**	$\Theta(a^n),\ a > 1$	Calcul récursif de la suite de Fibonacci

Classe de complexité

Classement des complexités :

$$\mathcal{O}(1) \subset \mathcal{O}(\log n) \subset \mathcal{O}(n \log n) \subset \mathcal{O}(n^2) \subset \mathcal{O}(2^n) \subset \mathcal{O}(n!)$$



1 La récursivité
- Exemple introductif
- Définition
- Itératif vs Récursif

2 Complexité
- Exemple du cadenas
- Définition
- Cas pratique

Concrètement avec un programme

Code

```
1 def somme(liste):
2
3     res = 0 # ?
4     taille = len(liste)
5
6   for i in range(taille) :
7     res += liste[i]
8
9   return res
```

$$\mathcal{O}(somme()) = ?$$

Concrètement avec un programme

Code

```
1  def somme(liste):
2
3      res = 0 # O(1)
4      taille = len(liste) # ?
5
6    for i in range(taille) :
7      res += liste[i]
8
9    return res
```

$$\mathcal{O}(somme()) = 1+?$$

Concrètement avec un programme

Code

```
1 def somme(liste):
2
3     res = 0 # O(1)
4     taille = len(liste) # O(1)
5
6   for i in range(taille) :
7     res += liste[i] # ?
8
9   return res
```

$$\mathcal{O}(somme()) = 1 + 1 + ?$$

Concrètement avec un programme

Code

```
1 def somme(liste):
2
3     res = 0 # O(1)
4     taille = len(liste) # O(1)
5
6   for i in range(taille) :
7     res += liste[i] # O(1)
8
9   return res # ?
```

$$\mathcal{O}(somme()) = 1 + 1 + 1 + ?$$

Concrètement avec un programme

Code

```
1  def somme(liste):
2
3      res = 0 # O(1)
4      taille = len(liste) # O(1)
5
6    for i in range(taille) : # ?
7      res += liste[i] # O(1)
8
9    return res # O(1)
```

$$\mathcal{O}(somme()) = 1 + 1 + 1 + 1 + ?$$

Concrètement avec un programme

Code

```
1  def somme(liste):
2
3      res = 0 # O(1)
4      n = len(liste) # O(1)
5
6    for i in range(n) : # O(n)
7      res += liste[i] # O(1)
8
9    return res # O(1)
```

$$\mathcal{O}(somme()) = 1 + 1 + 1 + 1 + n$$

pour n taille de la liste

Concrètement avec un programme

$$\mathcal{O}(somme()) = \mathcal{O}(1 + 1 + 1 + 1 + n)$$

$$\text{donc } \mathcal{O}(somme()) = \mathcal{O}(n + 4)$$

Hors, quand on établit une complexité, on cherche **un ordre de grandeur.**

Concrètement avec un programme

$$\mathcal{O}(somme()) = \mathcal{O}(1 + 1 + 1 + 1 + n)$$

$$\text{donc } \mathcal{O}(somme()) = \mathcal{O}(n + 4)$$

Hors, quand on établit une complexité, on cherche **un ordre de grandeur.**

On cherche à se rapporter à une classe de complexité !

Famille d'algorithmes	Notation
Algorithmes **constants**	$\Theta(1)$
Algorithmes **logarithmiques**	$\Theta(\log n)$
Algorithmes **linéaires**	$\Theta(n)$
Algorithmes **quasi-linéaires**	$\Theta(n \log n)$
Algorithmes **quadratiques**	$\Theta(n^2)$
Algorithmes **cubiques**	$\Theta(n^3)$
Algorithmes **polynomiaux**	$\Theta(n^p),\ p \geq 1$
Algorithmes **exponentiels**	$\Theta(a^n),\ a > 1$

Concrètement avec un programme

$$\mathcal{O}(somme()) = \mathcal{O}(n+4)$$

On cherche à se rapporter à une classe de complexité !

Famille d'algorithmes	Notation
Algorithmes **constants**	$\Theta(1)$
Algorithmes **logarithmiques**	$\Theta(\log n)$
Algorithmes **linéaires**	$\Theta(n)$
Algorithmes **quasi-linéaires**	$\Theta(n \log n)$
Algorithmes **quadratiques**	$\Theta(n^2)$
Algorithmes **cubiques**	$\Theta(n^3)$
Algorithmes **polynomiaux**	$\Theta(n^p),\ p \geq 1$
Algorithmes **exponentiels**	$\Theta(a^n),\ a > 1$

Donc, $\mathcal{O}(somme()) = \mathcal{O}(n)$

Bilan complexité

- La complexité d'un algorithme est estimée avec le grand O (*big O en anglais*), noté $\mathcal{O}$.
- La complexité d'un algorithme mesure le nombre d'opérations nécessaire à son exécution en temps ou en espace.
- Pour calculer le nombre d'opérations, on parcourt pas à pas l'algorithme et on compte le coût de chaque opération.
- On cherche un ordre de grandeur du nombre d'opérations. ex : $3n^2$ et $n^2/2$ sont tout les deux $\mathcal{O}(n^2)$